

Módulo 04 — Skills vs. Agentes

Cada slide da apresentação explicado em linguagem simples: termos, o que falar e o que responder.

1. Slide a slide

Slide a slide

Este guia explica cada slide da apresentação em linguagem simples: o que significa cada termo, o que falar e o que responder se perguntarem. Leia uma vez inteiro antes da aula; durante a aula, as notas do PPTX bastam.

Se só der tempo de entender uma coisa: uma **skill** é um manual reutilizável (instruções e documentos numa pasta) que diz **como** fazer uma tarefa. Um **agente** é um modelo que **decide sozinho** o próximo passo e usa ferramentas, num loop. Agente é mais caro e menos previsível, então só vale quando você não consegue escrever os passos de antemão.

Antes da aula (uns 10 minutos):

1. `pnpm -F @mentoria/curador m04:comparar:solucao` (~6 min): gera os números da comparação.
2. Rode `demo:skill` e `demo:agente` uma vez.
3. `demo:casos` é instantâneo (não usa modelo).

Slide 1 — Capa

Diga: *"Até aqui o curador seguia um caminho fixo no código. Hoje vamos empacotar o conhecimento dele e, depois, dar autonomia a ele. E decidir quando cada coisa vale a pena."*

Slide 2 — Hoje

Os objetivos. Não leia a lista.

Slide 3 — A frase da aula

Comece pelo mais simples que resolve. Skill empacota o *como*; agente decide o *que fazer a seguir*.

Autonomia: a capacidade de o sistema decidir sozinho o que fazer. É o que define um agente, e é o que o torna mais caro e menos previsível.

Slide 4 — Divisória "1 · Quatro abordagens"

A pergunta que organiza a aula: **quem decide os passos?**

Slide 5 — Prompt, skill, agente, decisão

As quatro opções, de forma simples:

- **Prompt:** um pedido bem escrito, numa chamada ao modelo. Ex.: "reescreva este texto em linguagem simples".
- **Skill:** um pacote reutilizável: uma pasta com instruções e documentos de apoio. Qualquer pessoa ou ferramenta pode usar o mesmo pacote.
- **Agente:** o modelo trabalha num **loop** (repetição): olha a situação, escolhe uma ação (por exemplo, consultar um sistema), vê o resultado, escolhe a próxima, até terminar.
- **Decisão estruturada:** o que vimos na Aula 2: o modelo escolhe uma opção de uma lista, com confiança.

Linhas da tabela:

- **Quem decide os passos:** só no agente é o modelo.
 - **Custo:** o agente faz várias chamadas, e não dá para saber quantas antes.
 - **Previsibilidade:** o agente pode seguir caminhos diferentes a cada vez.
 - **Avaliar:** no agente, além da resposta, você avalia a **trajetória** (o caminho que ele fez: o que consultou, quanto gastou).
-

Slide 6 — Workflow x agente

Distinção de um texto muito citado da Anthropic:

- **Workflow** (fluxo de trabalho): o **código** define o caminho. O modelo pode ser chamado várias vezes e usar ferramentas, mas sempre na ordem que o programador escreveu. Exemplos: encadear prompts (a saída de um vira entrada do outro), rotear (escolher qual prompt usar), paralelizar (rodar vários ao mesmo tempo).
- **Agente:** o **modelo** decide o caminho.

A citação: "otimizar chamadas únicas com recuperação e exemplos geralmente basta". Ou seja: antes de pensar em agente, tente um bom prompt com os dados certos.

Pergunta provável: *"Um chatbot que consulta o banco de dados é agente?"* Se o código decide quando consultar, é workflow. Se o modelo decide se consulta, o que consulta e quando parar, é agente.

Slide 7 — Divisória "2 · Framework de decisão"

Framework: aqui, um roteiro de perguntas para decidir.

Slide 8 — Quatro perguntas, nesta ordem

1. **Os passos mudam conforme o que se descobre e precisam de ferramentas?** → agente.
(Ex.: investigar um problema: você não sabe qual será a segunda consulta antes de ver o resultado da primeira.)
2. **A resposta é uma escolha de uma lista fixa, sempre do mesmo jeito?** → decisão estruturada.
3. **Vai ser usado várias vezes, por várias pessoas, ou precisa de muito conhecimento específico?** → skill.
4. **Nenhuma das anteriores?** → prompt.

Ferramenta (no contexto de agentes): uma função que o modelo pode pedir para executar: consultar um sistema, buscar um arquivo, enviar algo.

A ordem importa: uma decisão que precisa investigar com ferramentas é agente, porque a regra 1 vem antes.

Diga: é uma heurística (uma regra prática), não uma lei. Serve para começar pelo mais simples.

Slide 9 — O que muda ao virar agente

- **Custo variável:** depende do caminho que ele escolher.
- **Cobertura:** o agente pode decidir não ler algo. Às vezes economiza; às vezes perde coisa importante.
- **Trajetória:** pode mudar entre execuções.
- **Falhas novas:**
 - parar sem entregar o resultado (acabaram os passos);
 - repetir a mesma ação em loop;
 - inventar identificadores que não existem;
 - obedecer a instruções escondidas no que as ferramentas devolvem (a injeção da Aula 3, agora vinda de uma fonte).

Proteções (diga em voz alta): limite de passos, limite de custo, detectar repetição, **validar no código** o que o agente entrega, registrar o caminho e pedir aprovação humana antes de qualquer ação que não dá para desfazer.

Slide 10 — Divisória "3 · O curador como skill"

Passe direto.

Slide 11 — Uma skill é uma pasta

Mostre a pasta `projeto-final/solucao/skills/curar-noticia/`.

- **SKILL.md:** o arquivo principal. Começa com um cabeçalho (entre `---`) com:
 - `name`: o nome, igual ao da pasta;
 - `description`: o que a skill faz **e quando usar**.
- Depois do cabeçalho: as instruções, em texto.
- **references/:** documentos de apoio (aqui, o guia de rotulagem com as categorias).

Formato aberto (Agent Skills): a Anthropic criou esse formato em 2025 e publicou como padrão aberto. Várias ferramentas de agentes de código entendem a mesma pasta. Isso é o "reutilizável": você escreve uma vez e usa em vários lugares.

A parte mais importante é a `description`: é a única coisa que o agente lê antes de decidir se vai usar a skill. Se ela for vaga ("ajuda com notícias"), a skill nunca é usada.

Slide 12 — Divulgação progressiva

Rode `demo:skill`.

O agente não lê todas as skills inteiras (seria caro). Ele lê em camadas:

1. **Sempre:** só o nome e a descrição de cada skill. Aqui, ~79 tokens.
2. **Ao ativar:** quando a tarefa combina com a descrição, ele lê as instruções completas (~400 tokens).
3. **Sob demanda:** os documentos de apoio, só se precisar (~220 tokens).

Analogia: a lombada de um manual na estante. Você lê o título de todos; abre só o que serve; e consulta o apêndice só se precisar.

Bônus medido (conte esta história): na Aula 3, a notícia n13 ("startup promete nunca errar") passou por todos os guardrails e foi publicada. Na skill, escrevemos um exemplo de "promessa sem detalhes" e a regra "na dúvida, baixe a confiança". Resultado: a n13 agora vai para revisão. É o ciclo da Aula 3: mudou o prompt, mediu, melhorou.

Slide 13 — Divisória "4 · O curador como agente"

Passe direto.

Slide 14 — O loop

Rode `demo:agente` (~2 min) ou explique pela trajetória.

O agente recebe o objetivo ("monte a seleção da semana") e quatro ferramentas:

- **listarFontes:** mostra as 10 fontes de notícias com uma descrição de cada;
- **lerFonte:** mostra os títulos das notícias de uma fonte;
- **avaliarNoticias:** classifica as notícias com a skill e aplica os guardrails da Aula 3;
- **entregarSelecao:** entrega a lista final.

Ele decide quais fontes ler e quais notícias avaliar.

"O agente propõe; o código decide": depois que o agente entrega a lista, o código confere. Se ele entregar uma notícia que nunca avaliou (por exemplo, um código inventado) ou que a avaliação mandou não publicar, o código recusa.

Slide 15 — Pipeline x agente: os números

Pipeline: o caminho fixo (avaliar todas as notícias com a skill, uma por uma).

- **Pipeline + skill:** escolheu as 27 notícias certas, com 40 chamadas ao modelo, em 106 segundos.
- **Agente:** escolheu 24 (todas corretas), mas **perdeu 3**, fez 43 chamadas e demorou de 114 a 132 segundos.

O que aconteceu:

- **Acerto:** pulou a fonte "variedades" (entretenimento e esportes), que não tinha nada relevante. É exatamente a promessa de um agente: economizar escolhendo o que ler.

- **Erro:** leu 9 fontes (35 notícias) e montou **uma lista única** com 31 notícias para avaliar, esquecendo 4. É o mesmo tipo de problema do "perdido no meio" da Aula 1: listas longas no contexto fazem o modelo deixar coisas para trás.
- As duas execuções seguiram exatamente o mesmo caminho (temperatura 0).

Precisão e recall (Aula 3): precisão 100% = tudo que ele escolheu era bom; recall 89% = das boas, achou 89%.

A conclusão honesta (diga): aqui o agente não ganhou. Poucas fontes, baratas de ler, critério fixo: o framework diria "pipeline + skill", e os números concordam. O agente faria sentido com centenas de fontes caras, ou quando o próximo passo depende do que se descobre.

Slide 16 — Divisória "5 · Os 4 casos"

Passe direto.

Slide 17 — Classifique

Leia os 4 casos. **Não revele as respostas.** Peça para a turma pensar e votar na discussão (por exemplo, no chat: 1-P, 2-D...).

Slide 18 — Divisória "Discussão"

15 minutos. Comece pela votação dos casos.

Slide 19 — Respostas

Depois da votação, mostre (ou rode `demo:casos --revelar`):

1. **Comunicado** → **prompt**. Texto para pessoas, feito uma vez, sem dados externos. Montar qualquer coisa além de um bom prompt seria custo sem retorno.
2. **Tickets** → **decisão estruturada**. É escolher uma de 8 filas, sempre do mesmo jeito, 30 mil vezes por dia. Uma decisão tipada com confiança é barata, rápida e previsível; a confiança decide o que vai para uma pessoa.
3. **Contratos** → **skill**. Os passos são fixos (seguir o checklist), mas o conhecimento é grande e precisa ser o mesmo em todos os lugares. A skill empacota checklist e cláusulas-modelo; ninguém precisa de autonomia.

4. **Incidente** → **agente**. Não dá para saber de antemão quais consultas fazer: cada uma depende do que a anterior mostrou. Com limites, e com uma pessoa aprovando qualquer ação que mude o sistema.

Se alguém discordar: ótimo, peça o argumento. Exemplo de boa discordância: "o caso 2 poderia ser uma skill usada por um agente de atendimento". Resposta: a decisão em si é estruturada; ela pode viver dentro de outra coisa.

Slide 20 — Para conversar (com respostas esperadas)

1. **"Quando o agente ganharia do pipeline?"** Com muitas fontes caras de ler (centenas, pagas, lentas), quando escolher o que ler economiza de verdade; ou quando o próximo passo depende do conteúdo (por exemplo, seguir links de uma notícia para checar a fonte original).
2. **"Que caso do seu trabalho pede agente?"** Deixe as pessoas darem exemplos e aplique a pergunta 1: os passos mudam conforme o que se descobre e precisam de ferramentas? Se não, desça para algo mais simples.
3. **"Skill x prompt longo num arquivo?"** A skill tem formato padrão (outras ferramentas entendem), descrição para ser ativada sozinha, documentos carregados só quando precisa e pode ter scripts. Um prompt num arquivo é só texto que alguém precisa colar.
4. **"O que o código deve checar?"** Que tudo que o agente entregou existe e foi verificado por uma ferramenta; que respeita as regras (guardrails); custo e passos dentro do limite; e, para ações que mudam algo, aprovação humana.
5. **"Como avaliar um agente além da resposta?"** Pela trajetória: quantos passos, quanto custou, quais ferramentas usou, se pulou algo importante, se repetiu ações, se a resposta muda entre execuções. E, como na Aula 3, com um conjunto de casos e um portão.

Se o tempo apertar: casos, depois 1 e 4.

Slide 21 — Para a próxima aula

- **Exercício:** transformar o framework de decisão numa função e escrever um agente **sem biblioteca**: um loop simples, em que o "modelo" é uma função de teste. O desafio adiciona proteções (parar se o agente repetir a mesma ação ou estourar um orçamento).
 - **Projeto final, etapa M4:** escrever a skill, as ferramentas e o agente, e rodar a comparação.
-

Slide 22 — Próxima aula

Gancho: **sistemas multiagênticos** (vários agentes, cada um com um papel: coletor, classificador, redator, revisor) e **MCP** (*Model Context Protocol*, um padrão para expor ferramentas e dados a qualquer agente).

Glossário rápido

Termo	Em uma frase
Prompt	um pedido bem escrito ao modelo
Skill	pasta reutilizável com instruções e documentos para uma tarefa
Agent Skills	formato aberto de skills, entendido por várias ferramentas
SKILL.md	o arquivo principal da skill (cabeçalho + instruções)
Frontmatter	o cabeçalho entre <code>---</code> com nome e descrição
Divulgação progressiva	carregar só o necessário, em camadas
Agente	modelo num loop que decide o próximo passo e usa ferramentas
Ferramenta	função que o modelo pode pedir para executar
Tool calling	o modelo devolve um pedido estruturado de ferramenta, o código executa
Loop	repetição: decidir → executar → ver resultado → decidir
Workflow	caminho definido no código (mesmo que use o modelo várias vezes)
Autonomia	o sistema decidir sozinho o que fazer
Trajetória	o caminho que o agente fez (ações, ordem, custo)
Pipeline	sequência fixa de etapas
Decisão estruturada	escolha tipada de uma lista, com confiança
Heurística	regra prática para decidir, não uma lei
Limite de passos / orçamento	proteções que param o agente
ReAct	método que intercala raciocínio e ação num loop
MCP	padrão para expor ferramentas e dados a agentes (próxima aula)